



UNIVERSIDAD
CATÓLICA DE
TEMUCO

DEPARTAMENTO DE
INGENIERÍA INFORMÁTICA
FACULTAD DE INGENIERÍA

Universidad Católica de Temuco

Facultad de Ingeniería

Ingeniería Civil en Informática

Evaluación 3: Sistema de Gestión de Restaurante con ORM

Asignatura: Programación II

Docente: Guido Mellado

Ayudantes: Fernando Valdés, Joaquín Cantero

Integrantes:

Gabriel Gutiérrez, Ailyn Melillán, Nicolás Cayumán

Noviembre 2025

Índice

1. Introducción	2
2. Comparación entre Evaluación 2 y Evaluación 3	2
2.1. Mejoras más relevantes respecto a EV2	2
3. Estructura General del Proyecto	3
4. Modelo de Datos y ORM	3
4.1. Diagrama MER	3
4.2. Definición de modelos ORM	3
5. CRUDs (Create, Read, Update, Delete)	4
5.1. Ejemplo: CRUD de Menús	4
6. Interfaz Gráfica (CustomTkinter)	4
7. Programación Funcional	5
7.1. Uso de <code>map</code>	5
7.2. Uso de <code>filter</code>	5
7.3. Uso de <code>reduce</code>	5
7.4. Uso de <code>lambda</code> en eventos	5
8. Generación de Boleta (Patrón Facade)	5
8.1. Arquitectura del Facade	5
9. Flujo Completo del Sistema	6
10. Conclusiones	7

1. Introducción

El presente documento expone el desarrollo del **Sistema de Gestión de Clientes, Pedidos, Menús e Ingredientes para un Restaurante**, correspondiente a la Evaluación 3.

Este sistema representa un avance significativo respecto a la Evaluación 2, agregando:

- Persistencia profesional mediante **SQLAlchemy ORM**.
- Implementación completa de **CRUDs** modularizados.
- Manejo de **relaciones complejas** entre entidades (1:N y N:N).
- Interfaz gráfica profesional usando **CustomTkinter**.
- Programación funcional: **lambda**, **map**, **filter**, **reduce**.
- Generación de PDF: carta del menú y boleta del pedido.
- Gráficos estadísticos integrados con Matplotlib.
- Uso de patrones de diseño: **DTO** y **Facade**.

El objetivo del informe es describir detalladamente la arquitectura interna, el flujo del programa, las estructuras de datos, la interacción entre módulos y el cumplimiento de criterios de la pauta.

2. Comparación entre Evaluación 2 y Evaluación 3

2.1. Mejoras más relevantes respecto a EV2

La Evaluación 3 constituye una reingeniería total del proyecto. Las mejoras más importantes son:

- **Migración completa a ORM:** desde clases en memoria (EV2) hacia un sistema persistente con SQLAlchemy (EV3).
- **Creación de CRUDs separados:** en EV2 toda la lógica estaba unida; en EV3 se modulariza según entidad.
- **Implementación de DTOs:** permiten desacoplar la interfaz gráfica del ORM.
- **Control real de stock:** se descuenta según recetas (N:N), no existía en EV2.
- **Interfaz moderna y multipestaña:** EV2 tenía una GUI mínima.
- **PDFs:** boleta y carta del restaurante.
- **Gráficos estadísticos:** inexistentes en EV2.

3. Estructura General del Proyecto

La estructura del proyecto es modular:

- **app.py**: flujo principal del programa e interfaz gráfica.
- **database.py**: inicialización del ORM y la base de datos.
- **models.py**: modelos ORM (Cliente, Menú, Ingrediente, Pedido, Detalle, asociaciones).
- Carpeta **crud/**:
 - **cliente_crud.py**
 - **crud_pedido.py**
 - **crud_detallepedido.py**
 - **ingrediente_crud.py**
 - **menu_crud.py**
- **BoletaFacade.py**: encapsula el proceso completo de generación de boletas.
- **ElementoMenu.py** y **Ingrediente.py**: DTOs.
- **graficos.py**: extracción de datos y construcción de gráficos.

Cada módulo cumple una única responsabilidad, siguiendo los principios SOLID.

4. Modelo de Datos y ORM

4.1. Diagrama MER

```
erDiagram
    CLIENTE ||--o{ PEDIDO : genera
    PEDIDO ||--o{ DETALLE_PEDIDO : contiene
    MENU ||--o{ DETALLE_PEDIDO : vendido
    MENU ||--o{ MENU_INGREDIENTE : receta
    INGREDIENTE ||--o{ MENU_INGREDIENTE : requerido
```

4.2. Definición de modelos ORM

Ejemplo de un modelo utilizado:

```

class MenuIngrediente(Base):
    __tablename__ = "menu_ingredientes"

    id = Column(Integer, primary_key=True)
    menu_id = Column(Integer, ForeignKey("menus.id"))
    ingrediente_id = Column(Integer, ForeignKey("ingredientes.id"))
    cantidad_requerida = Column(Float)

    menu = relationship("Menu", back_populates="ingredientes_asociados")
    ingrediente = relationship("Ingrediente")

```

Este diseño permite controlar el stock de cada menú de forma precisa.

5. CRUDs (Create, Read, Update, Delete)

5.1. Ejemplo: CRUD de Menús

```

def create_menu(self, db, nombre, precio, icono):
    nuevo = Menu(nombre=nombre, precio=precio, icono=icono)
    db.add(nuevo)
    db.commit()
    db.refresh(nuevo)
    return nuevo

```

Aspectos importantes:

- `db.add()` + `db.commit()` permite persistir cambios.
- `db.refresh()` obtiene valores actualizados del ORM.
- Todas las operaciones CRUD están aisladas en un módulo específico.

6. Interfaz Gráfica (CustomTkinter)

El archivo `app.py` organiza todas las vistas en formato de pestañas:

```

self.tabview.add("Pedido")
self.tabview.add("Ingredientes")
self.tabview.add("Menús")
self.tabview.add("Clientes")
self.tabview.add("Gráficos")
self.tabview.add("Boleta")

```

Cada pestaña interactúa con los CRUDs y DTOs.

7. Programación Funcional

7.1. Uso de map

```
nombres_clientes = list(map(
    lambda c: f"{c.id} - {c.nombre}",
    clientes
))
```

7.2. Uso de filter

```
menus_disponibles = list(filter(
    lambda m: self.menu_crud.esta_disponible(self.db_session, m),
    menus_db
))
```

7.3. Uso de reduce

```
total = reduce(
    lambda subtotal, item: subtotal + item.precio * item.cantidad,
    self.pedido.menus, 0.0
)
```

7.4. Uso de lambda en eventos

```
tarjeta.bind("<Enter>", lambda e: tarjeta.configure(border_color="red"))
```

8. Generación de Boleta (Patrón Facade)

8.1. Arquitectura del Facade

```
class BoletaFacade:
    def procesar_pedido(self, pedido_dto):
        pedido_orm = self.pedido_crud.crear_pedido(self.db, pedido_dto)
        self.detalle_crud.crear_detalles(self.db, pedido_orm, pedido_dto)
        self.actualizar_stock(pedido_orm)
        return self.generar_pdf(pedido_orm)
```

Responsabilidades encapsuladas:

- Registrar el pedido.

- Registrar detalles.
- Descontar stock.
- Generar boleta en PDF.

9. Flujo Completo del Sistema

1. Inicialización

```
create_db_and_tables()  
self.db = SessionLocal()
```

2. Construcción de pestañas y DTOs

```
self.pedido = Pedido()
```

3. Agregar menú al carrito

```
self.pedido.menus.append(menu_dto)
```

4. Validación de stock

```
if not self.menu_crud.esta_disponible(self.db, menu_orm):  
    return
```

5. Cálculo del total

```
total = reduce(lambda t, m: t + m.precio*m.cantidad, self.pedido.menus, 0)
```

6. Persistencia del pedido vía Facade

```
ruta_pdf = facade.procesar_pedido(self.pedido)
```

7. Visualización de la boleta en PDF

10. Conclusiones

El sistema final cumple todos los requisitos establecidos por la pauta de la Evaluación 3, integrando:

- Interfaz gráfica robusta.
- ORM profesional con SQLAlchemy.
- CRUDs completos y modularizados.
- Programación funcional aplicada.
- Flujo completo del proceso de compra.
- Generación de PDFs.
- Estadísticas gráficas.

El avance respecto a la Evaluación 2 es evidente tanto en arquitectura como en funcionalidad y calidad del código.